pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 1

Operaciones CRUD 1 - Recursos

Daniel Trabas

pontIA

Índice

Objetivo 3

Operaciones CRUD 1: Fundamentos de Consulta 3

PontIA

Objetivo

Este documento técnico sirve como base y guía de apoyo para la generación de material audiovisual dirigido a alumnos con conocimientos en IT. En él se detallan los conceptos fundamentales de las operaciones de lectura (CRUD) en PostgreSQL, basándonos en los esquemas de la base de datos de ejemplo demodb.

Operaciones CRUD 1: Fundamentos de Consulta

SELECT y FROM

En el entorno de las bases de datos relacionales, la obtención de información se realiza mediante consultas. En PostgreSQL, la estructura fundamental para esto es la sentencia SELECT. Según la documentación oficial, la instrucción "SELECT, TABLE, WITH — recupera filas de una tabla o vista".

De forma general, una consulta se compone de dos partes obligatorias:

1. **SELECT:** Define qué columnas o datos exactos queremos extraer y mostrar.
2. **FROM:** Especifica la tabla o tablas de origen de donde provienen esos datos.

Esquemas y Alias de Tabla

PostgreSQL organiza las tablas dentro de agrupaciones lógicas llamadas esquemas. Un esquema funciona como un espacio de nombres (namespace) donde residen las tablas; por defecto, se utiliza el esquema bookings. En el caso de nuestra base de datos llamada demodb, el esquema que utilizaremos será bookings.

Cuando la consulta incluye múltiples tablas, es una excelente práctica usar alias de tabla. Un alias es un nombre temporal y corto que le damos a una tabla en la cláusula FROM para simplificar la escritura de la consulta y evitar ambigüedades.

Limitadores de Resultados (Sin WHERE)

A menudo no queremos recuperar toda la tabla, sino sólo una porción. Podemos restringir el número de filas devueltas sin filtrar por condiciones utilizando modificadores al final de la consulta:

- **LIMIT:** Especifica el número máximo de filas a devolver.
- **OFFSET:** Indica cuántas filas saltar antes de empezar a devolver resultados. "Finalmente, las filas que caen fuera del rango especificado por LIMIT y OFFSET se descartan".
- **FETCH:** Funciona de manera similar a LIMIT, pero pertenece al estándar SQL moderno.

Ejemplos prácticos sobre demodb:

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```sql
-- 1. Recuperar todos los datos de la cabecera de reservas
SELECT bk.*
FROM    bookings.bookings bk
LIMIT   100;

-- 2. Uso de alias de tabla para simplificar la consulta de aeropuertos
SELECT ai.airport_code,
       ai.airport_name
FROM    bookings.airports ai;

-- 3. Limitar el resultado a los primeros 10 aviones registrados
SELECT ap.airplane_code,
       ap.model
FROM    bookings.airplanes ap;

-- 4. Mostrar los primeros 5 pasajeros registrados en la tabla de tickets
SELECT tk.ticket_no,
       tk.passenger_name
FROM    bookings.tickets tk
LIMIT   5;

-- 4.1 Paginación: Saltar los primeros 5 billetes y mostrar los siguientes 5
SELECT tk.ticket_no,
       tk.passenger_name
FROM    bookings.tickets tk
OFFSET 5 ROWS
FETCH FIRST 5 ROWS ONLY;

-- 5. Uso de DISTINCT y ORDER BY para obtener valores únicos en una columna
SELECT DISTINCT ai.country
FROM  bookings.airports ai
ORDER BY ai.country;
```

pontIA

WHERE

La cláusula WHERE nos permite aplicar filtros lógicos a nuestras consultas. "Cualquier fila que no satisfaga esta condición será eliminada de la salida". El filtro evalúa cada fila individualmente y solo devuelve aquellas donde la condición sea evaluada como verdadera.

Escenarios de uso:

- Buscar el perfil de un usuario específico por su ID.
- Encontrar productos que sean más caros que cierto límite.
- Filtrar pedidos que ya han sido enviados.

Ejemplos prácticos sobre demodb:

```sql
-- 1. Filtrar un aeropuerto por un valor de texto exacto (país)
SELECT ai.airport_code,
       ai.airport_name
FROM   bookings.airports ai
WHERE  ai.country = 'Spain';

-- 2. Filtrar utilizando operadores matemáticos en el coste de las reservas
SELECT bk.book_ref,
       bk.total_amount
FROM   bookings.bookings bk
WHERE  bk.total_amount > 1000.00;

-- 3. Combinar condiciones lógicas con AND para vuelos a tiempo en una ruta u
usando ORDER BY para mostrar resultados de forma descendente
SELECT fl.flight_id,
       fl.scheduled_departure
FROM   bookings.flights fl
WHERE  fl.status = 'Arrived'
  AND  fl.route_no = 'PG0139'
ORDER BY fl.flight_id DESC;

-- 4. Filtrar y limitar los resultados simultáneamente en segmentos de vuelo
SELECT sg.ticket_no,
       sg.price
FROM   bookings.segments sg
WHERE  sg.fare_conditions = 'Business'
LIMIT  5;
```

FILTROS AVANZADOS

Además de los operadores básicos (=, <, >), PostgreSQL ofrece filtros avanzados dentro del WHERE para búsquedas más complejas.

BETWEEN

Evalúa si un valor se encuentra dentro de un rango (incluyendo los extremos). Se puede utilizar para hacer un filtrado entre valores como por ejemplo, filtrar ventas realizadas en un trimestre específico:

`… WHERE BETWEEN transaction_date BETWEEN '20260101' AND '20260331'`

IN

Comprueba si un valor coincide con alguno de los elementos de una lista. Se utiliza por ejemplo para hacer una búsqueda de algún valor (aeropuertos) que puedan estar en tres países concretos:

`… WHERE country IN ('Spain', 'United States', 'New Zealand')`

LIKE / ILIKE

Permite buscar patrones de texto usando comodines como % (cualquier cadena) o _ (un solo carácter). Un ejemplo es tratar de encontrar a un usuario con un apellido concreto que pueda estar dentro de una cadena de nombre completo:

`… WHERE customer_name LIKE '%Ferr_n%'`

IS NULL

Verifica si un campo carece de valor (es nulo). Lo podremos utilizar por ejemplo, para identificar pedidos a los que aún no se les ha asignado un repartidor:

`… WHERE order_delivery_status IS NULL`

Ejemplos prácticos sobre demodb:

```
-- 1. Uso de BETWEEN para rangos de fechas en las reservas
SELECT bk.book_ref,
       bk.book_date
FROM  bookings.bookings bk
WHERE
      bk.book_date BETWEEN '2026-01-10' AND '2026-01-11';

-- 2. Uso de IN para listas de valores en ciudades de aeropuertos
SELECT ai.airport_name,
       ai.city
FROM  bookings.airports ai
WHERE   ai.city IN ('Madrid', 'Barcelona', 'Valencia');

-- 3. Uso de LIKE para buscar patrones de texto (pasajeros cuyo nombre empieza
por 'Garcia')
SELECT tk.passenger_name,
       tk.ticket_no
FROM  bookings.tickets tk
WHERE  tk.passenger_name LIKE '%Mor%s'
LIMIT
      200;

-- 4. Uso de IS NULL para encontrar vuelos sin salida real registrada (aún
pendientes)
SELECT fl.flight_id, fl.status
FROM  bookings.flights fl
WHERE  fl.actual_departure IS NULL
LIMIT
      100;
```

JOINS

En bases de datos relacionales, los datos suelen estar normalizados y divididos en múltiples tablas. En PostgreSQL se utiliza para combinar columnas de una o más tablas basándose en los valores de las columnas comunes entre las tablas relacionadas.

Tipos de JOINS

INNER JOIN

Incluyente-estricto: sólo devuelve las filas donde hay coincidencia en ambas tablas. Se puede utilizar por ejemplo, para ver solo los clientes que han realizado al menos un pedido.

LEFT JOIN

Incluyente izquierdo / Excluyente derecho: devuelve todas las filas de la tabla izquierda, y las coincidencias de la derecha. Si no hay coincidencia, rellena el lado derecho con valores NULL. Lo usaremos por ejemplo cuando hagamos un informe de todos los clientes registrados, indicando qué pedidos han hecho (incluso si no han hecho ninguno).

RIGHT JOIN

Incluyente derecho / Excluyente izquierdo: es Igual que el LEFT JOIN, pero invertido. Se usa en ocasiones excepcionales o cuando queremos tener el resultado del a tabla derecha intacta.

FULL OUTER JOIN

Incluyente total: devuelve las filas cuando hay una coincidencia en una de las dos tablas. Extrae los registros huérfanos de ambos lados. Es algo muy utilizado en auditorías para la validez de la consistencia entre dos tablas.

PontIA

Ejemplos prácticos sobre demodb:

pontIA

```sql
-- 1. INNER JOIN: Reservas y sus billetes cruzados por la referencia (book_ref)
SELECT bk.book_date,
       tk.passenger_name,
       tk.ticket_no
FROM  bookings.bookings bk
  INNER JOIN bookings.tickets tk ON bk.book_ref = tk.book_ref
WHERE   bk.book_date BETWEEN '202-01-10' AND '2026-01-11'
LIMIT
     10;

-- 2. LEFT JOIN: Todos los aviones y, si la tienen, la configuración de sus
asientos
SELECT ap.model,
        st.seat_no,
        st.fare_conditions
FROM  bookings.airplanes ap
  LEFT JOIN bookings.seats st ON ap.airplane_code = st.airplane_code;

-- 3. RIGHT JOIN: Todas las tarjetas de embarque y la información del billete
asociado por el número de embarque 44.
SELECT bp.boarding_no,
       tk.passenger_name
FROM  bookings.tickets tk
  RIGHT JOIN bookings.boarding_passes bp ON tk.ticket_no = bp.ticket_no
WHERE   bp.boarding_no = 44;

-- 3.1 RIGHT JOIN: Todas las tarjetas de embarque y la información del billete
asociado para el número de embarque 44.
-- Tendremos que usar la cláusula WHERE para filtrar el número de embarque, ya
que no podemos ponerlo en la condición del JOIN si queremos que se muestren
todas las tarjetas de embarque, incluso aquellas que no tienen billete
asociado.
-- Si no lo hacemos, tendremos que irnos a la posición 274 para ver el primer
resultado.
SELECT bp.boarding_no,
       tk.passenger_name
FROM  bookings.tickets tk
  RIGHT JOIN bookings.boarding_passes bp ON (tk.ticket_no = bp.ticket_no
    AND bp.boarding_no = 44)
LIMIT
     1000;

-- 3.2 RIGHT JOIN: Todas las tarjetas de embarque y la información del billete
asociado para el número de embarque 44.
-- Añadir la condición de filtrado en el WHERE para que se muestren todas las
tarjetas de embarque 44 sin el resto de resultados a NULL.
SELECT bp.boarding_no,
       tk.passenger_name
FROM  bookings.tickets tk
  RIGHT JOIN bookings.boarding_passes bp ON (tk.ticket_no = bp.ticket_no
    AND bp.boarding_no = 44)
WHERE tk.passenger_name IS NOT NULL
```

```
LIMIT
      1000;

-- 4. FULL OUTER JOIN: Rutas teóricas y vuelos programados cruzados por ruta
SELECT rt.route_no,
       fl.flight_id,
       fl.status
FROM  bookings.routes rt
  FULL OUTER JOIN bookings.flights fl ON rt.route_no = fl.route_no
ORDER by fl.route_no;
```

UNIONS

Mientras que los JOINS combinan columnas de diferentes tablas de forma horizontal, los operadores de conjuntos como UNION combinan los resultados de dos consultas distintas de forma vertical (añadiendo filas).

UNION efectivamente añade el resultado de query2 al resultado de query1. Además, elimina las filas duplicadas de su resultado a menos que se use UNION ALL

Escenarios de uso

- Consolidar reportes de bases de datos divididas (e.g., ventas históricas y ventas actuales).
- Crear una lista combinada de contactos extrayendo emails de la tabla de clientes y de la tabla de proveedores.

PontIA

Ejemplos prácticos sobre demodb:

pontIA

```sql
-- 1. UNION simple: Combina eliminando códigos duplicados entre tablas maestras
de aeropuertos
SELECT ai.airport_code
FROM    bookings.airports ai
UNION
SELECT airport_code
FROM    bookings.airports_data aid;

-- 2. UNION ALL: Combina manteniendo todos los registros de códigos de aviones
(más rápido)
SELECT ap.airplane_code
FROM    bookings.airplanes ap
UNION ALL
SELECT apd.airplane_code
FROM    bookings.airplanes_data apd;

-- 3. UNION con literales: Clasificar el origen del identificador
SELECT tk.ticket_no AS identificador,
       'Billete' AS tipo_registro
FROM    bookings.tickets tk
WHERE   tk.passenger_id LIKE 'JP%'
  AND   tk.outbound = 'Y'
UNION
SELECT bk.book_ref AS identificador,
       'Reserva' AS tipo_registro
FROM    bookings.bookings bk
WHERE   bk.book_ref LIKE 'JP%';

-- 4. UNION aplicando condiciones previas: Agrupar vuelos con incidencias
SELECT fl.flight_id,
       fl.status
FROM    bookings.flights fl
WHERE   fl.status = 'Cancelled'
UNION
SELECT fl.flight_id,
       fl.status
FROM    bookings.flights fl
WHERE   fl.status = 'Delayed';

-- 5. UNION con ordenación: Agrupar vuelos con incidencias y ordenar por estado
SELECT ft.flight_id,
       ft.status
FROM    ( SELECT fl.flight_id,
                 fl.status
          FROM    bookings.flights fl
          WHERE   fl.status = 'Cancelled'
          UNION
          SELECT fl.flight_id,
                 fl.status
          FROM    bookings.flights fl
          WHERE   fl.status = 'Delayed'
```

```
        ) AS ft
ORDER BY ft.status;
```

pontIA